Lab 3: Data Preparation

CPE232 Data Models

For this lab, we will learn how to perform data preparation for data analysis after receiving the raw data from the data source.

[1] Reviews on Pandas

1.1) Discover

- methods to explore and understand your DataFrame

In [1]:
```
import pandas as pd

df = pd.read_csv('nss15.csv')
df.head()
```

Out[1]:

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagnosis	b
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	57	
1	150734723	7/6/2015	83.2157	S	36	Male	White	57	
2	150817487	8/2/2015	74.8813	L	20	Female	NaN	71	
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	71	
4	150721694	7/4/2015	74.8813	L	88	Female	Other	62	

In [2]:
```
# see the shape of the dataframe
```

```
print(df.shape)
```

```
(334839, 12)
```

In [3]:
```
# seeing the summary of the dataframe
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 334839 entries, 0 to 334838
Data columns (total 12 columns):
 #   Column         Non-Null Count   Dtype
---  ------         --------------   -----
 0   caseNumber     334839 non-null  int64
 1   treatmentDate  334839 non-null  object
 2   statWeight     334839 non-null  float64
 3   stratum        334839 non-null  object
 4   age            334839 non-null  int64
 5   sex            334837 non-null  object
 6   race           205014 non-null  object
 7   diagnosis      334839 non-null  int64
 8   bodyPart       334839 non-null  int64
 9   disposition    334839 non-null  int64
 10  location       334839 non-null  int64
 11  product        334839 non-null  int64
dtypes: float64(1), int64(7), object(4)
memory usage: 30.7+ MB
None
```

In [4]:
```
# seeing the stats of the column in dataframe
df.describe()
```

Out[4]:

	caseNumber	statWeight	age	diagnosis	bodyPart	d
count	3.348390e+05	334839.000000	334839.000000	334839.000000	334839.000000	3348
mean	1.510271e+08	39.343028	31.385451	60.154591	64.374192	
std	1.720330e+06	34.142933	26.105098	6.170699	24.002331	
min	1.501032e+08	4.965500	0.000000	41.000000	0.000000	
25%	1.504405e+08	15.059100	10.000000	57.000000	35.000000	
50%	1.507358e+08	15.776200	23.000000	59.000000	75.000000	
75%	1.510231e+08	74.881300	51.000000	64.000000	82.000000	
max	1.603418e+08	97.923900	107.000000	74.000000	94.000000	

1.2) Selecting variables

- select specific columns from the DataFrame to create a new DataFrame with only those columns

In [5]:
```
# select columns based on the data type
df.select_dtypes(include=['number'])
```

Out[5]:

	caseNumber	statWeight	age	diagnosis	bodyPart	disposition	location	proc
0	150733174	15.7762	5	57	33	1	9	1
1	150734723	83.2157	36	57	34	1	1	1
2	150817487	74.8813	20	71	94	1	0	3
3	150717776	15.7762	61	71	35	1	0	
4	150721694	74.8813	88	62	75	1	0	1
...	...	...	...	...	...	...	...	
334834	150739278	15.0591	7	59	76	1	1	1
334835	150733393	5.6748	3	68	85	1	0	1
334836	150819286	15.7762	38	71	79	1	0	3
334837	150823002	97.9239	38	59	82	1	1	
334838	150723074	49.2646	5	57	34	1	9	3

334839 rows × 8 columns

In [6]:
```
# select column by .loc
df.loc[:6,'treatmentDate':'diagnosis']
```

Out[6]:

	treatmentDate	statWeight	stratum	age	sex	race	diagnosis
0	7/11/2015	15.7762	V	5	Male	NaN	57
1	7/6/2015	83.2157	S	36	Male	White	57
2	8/2/2015	74.8813	L	20	Female	NaN	71
3	6/26/2015	15.7762	V	61	Male	NaN	71
4	7/4/2015	74.8813	L	88	Female	Other	62
5	7/2/2015	5.6748	C	1	Female	White	71
6	6/8/2015	15.7762	V	25	Male	Black	51

In [7]:
```
# select row by .iloc
df.iloc[0:5]
```

Out[7]:

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagnosis	b
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	57	
1	150734723	7/6/2015	83.2157	S	36	Male	White	57	
2	150817487	8/2/2015	74.8813	L	20	Female	NaN	71	
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	71	
4	150721694	7/4/2015	74.8813	L	88	Female	Other	62	

1.3) Filtering the data

In [8]:
```
# filter rows based on the condition
df[df['age'] > 50]
```

Out[8]:

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagno
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	
4	150721694	7/4/2015	74.8813	L	88	Female	Other	
7	150704114	6/14/2015	83.2157	S	53	Male	White	
8	150736558	7/16/2015	83.2157	S	98	Male	Black	
16	150901411	8/27/2015	83.2157	S	65	Female	White	
...	...	...	...	...	...	...	...	
334811	150702215	6/27/2015	15.7762	V	51	Female	NaN	
334815	151100368	10/28/2015	83.2157	S	85	Female	NaN	
334819	150528367	1/13/2015	49.2646	M	85	Female	NaN	
334826	150648619	6/17/2015	15.7762	V	52	Female	White	
334829	150633526	4/4/2015	49.2646	M	51	Female	NaN	

85235 rows × 12 columns

In [9]:
```
# filter coloum based on column name
df.filter(like='age')
```

Out[9]:

	age
0	5
1	36
2	20
3	61
4	88
...	...
334834	7
334835	3
334836	38
334837	38
334838	5

334839 rows × 1 columns

1.4) Sorting

- Sort the DataFrame by its index based on column

In [10]:
```
# sort the dataframe based on column name and ascending order
df.sort_values(by='statWeight', ascending=False)
```

Out[10]:

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagno
275174	150343700	3/9/2015	97.9239	M	48	Male	NaN	
36	151029422	10/6/2015	97.9239	M	37	Male	White	
334806	150612491	5/29/2015	97.9239	M	18	Female	White	
334810	150725804	7/8/2015	97.9239	M	33	Female	Black	
275161	150450816	4/13/2015	97.9239	M	24	Male	White	
...	...	...	...	...	...	...	...	
44011	160222258	12/29/2015	4.9655	C	2	Female	Other	
325320	151213065	11/29/2015	4.9655	C	16	Female	White	
43891	160113865	12/28/2015	4.9655	C	4	Male	White	
43628	151130111	11/9/2015	4.9655	C	13	Male	Black	
43523	151139237	11/16/2015	4.9655	C	2	Female	Black	

334839 rows × 12 columns

In [11]:
```
# sort the index of the dataframe
df.sort_index()
```

Out[11]:

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagno
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	
1	150734723	7/6/2015	83.2157	S	36	Male	White	
2	150817487	8/2/2015	74.8813	L	20	Female	NaN	
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	
4	150721694	7/4/2015	74.8813	L	88	Female	Other	
...	...	...	...	...	...	...	...	
334834	150739278	5/31/2015	15.0591	V	7	Male	NaN	
334835	150733393	7/11/2015	5.6748	C	3	Female	Black	
334836	150819286	7/24/2015	15.7762	V	38	Male	NaN	
334837	150823002	8/8/2015	97.9239	M	38	Female	White	
334838	150723074	6/20/2015	49.2646	M	5	Female	White	

334839 rows × 12 columns

1.5) Add/Remove

- This section shows how to manipulate the DataFrame's structure

In [12]:
```
# Dropping the column
df.drop(columns=['disposition'])
```

Out[12]:

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagno
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	
1	150734723	7/6/2015	83.2157	S	36	Male	White	
2	150817487	8/2/2015	74.8813	L	20	Female	NaN	
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	
4	150721694	7/4/2015	74.8813	L	88	Female	Other	
...	...	...	...	...	...	...	...	
334834	150739278	5/31/2015	15.0591	V	7	Male	NaN	
334835	150733393	7/11/2015	5.6748	C	3	Female	Black	
334836	150819286	7/24/2015	15.7762	V	38	Male	NaN	
334837	150823002	8/8/2015	97.9239	M	38	Female	White	
334838	150723074	6/20/2015	49.2646	M	5	Female	White	

334839 rows × 11 columns

In [13]:
```
# Adding column and create into a new column
df.assign(new_column=df['diagnosis'] + df['bodyPart'])
```

Out[13]:

	caseNumber	treatmentDate	statWeight	stratum	age	sex	race	diagno
0	150733174	7/11/2015	15.7762	V	5	Male	NaN	
1	150734723	7/6/2015	83.2157	S	36	Male	White	
2	150817487	8/2/2015	74.8813	L	20	Female	NaN	
3	150717776	6/26/2015	15.7762	V	61	Male	NaN	
4	150721694	7/4/2015	74.8813	L	88	Female	Other	
...	...	...	...	...	...	...	...	
334834	150739278	5/31/2015	15.0591	V	7	Male	NaN	
334835	150733393	7/11/2015	5.6748	C	3	Female	Black	
334836	150819286	7/24/2015	15.7762	V	38	Male	NaN	
334837	150823002	8/8/2015	97.9239	M	38	Female	White	
334838	150723074	6/20/2015	49.2646	M	5	Female	White	

334839 rows × 13 columns

In [14]:
```
# Removing the column and assigning it to a new variable
ages = df.pop('age')
```

1.6) Clean missing

- to remove rows with missing values or replace missing values with a specified value

In [15]:
```
# replaceing the missing values with a specified value
df.fillna(value=0)
```

Out[15]:

	caseNumber	treatmentDate	statWeight	stratum	sex	race	diagnosis	b
0	150733174	7/11/2015	15.7762	V	Male	0	57	
1	150734723	7/6/2015	83.2157	S	Male	White	57	
2	150817487	8/2/2015	74.8813	L	Female	0	71	
3	150717776	6/26/2015	15.7762	V	Male	0	71	
4	150721694	7/4/2015	74.8813	L	Female	Other	62	
...	...	...	...	...	...	...	...	
334834	150739278	5/31/2015	15.0591	V	Male	0	59	
334835	150733393	7/11/2015	5.6748	C	Female	Black	68	
334836	150819286	7/24/2015	15.7762	V	Male	0	71	
334837	150823002	8/8/2015	97.9239	M	Female	White	59	
334838	150723074	6/20/2015	49.2646	M	Female	White	57	

334839 rows × 11 columns

In [16]:
```
# Remove the rows with missing values
df.dropna()
```

Out[16]:

	caseNumber	treatmentDate	statWeight	stratum	sex	race	diagnosis	b
1	150734723	7/6/2015	83.2157	S	Male	White	57	
4	150721694	7/4/2015	74.8813	L	Female	Other	62	
5	150721815	7/2/2015	5.6748	C	Female	White	71	
6	150713483	6/8/2015	15.7762	V	Male	Black	51	
7	150704114	6/14/2015	83.2157	S	Male	White	57	
...	...	...	...	...	...	...	...	
334830	150628863	6/8/2015	15.7762	V	Female	White	64	
334831	150607637	5/22/2015	5.6748	C	Female	Black	59	
334835	150733393	7/11/2015	5.6748	C	Female	Black	68	
334837	150823002	8/8/2015	97.9239	M	Female	White	59	
334838	150723074	6/20/2015	49.2646	M	Female	White	57	

205014 rows × 11 columns

---

[2] Data Cleaning and Preparation

.isnull, .dropna, .fillna

2.1) checking

In [17]:
```
df.columns
```

Out[17]:
```
Index(['caseNumber', 'treatmentDate', 'statWeight', 'stratum', 'sex', 'race',
       'diagnosis', 'bodyPart', 'disposition', 'location', 'product'],
      dtype='object')
```

In [18]:
```
# isnull checking
df.isnull().sum()
```

Out[18]:
```
caseNumber            0
treatmentDate         0
statWeight            0
stratum               0
sex                   2
race             129825
diagnosis             0
bodyPart              0
disposition           0
location              0
product               0
dtype: int64
```

In [19]:
```
# percentage of missing values for the race
df.race.isnull().sum()/df.shape[0]*100
```

Out[19]:
```
np.float64(38.772365226272925)
```

In [20]:
```
df.shape[0]
```

Out[20]:
```
334839
```

2.2) Drop column

In [21]:
```
# remove column by using
df = df.drop(columns=['race'])
```

In [22]:
```
df.head()
```

Out[22]:

	caseNumber	treatmentDate	statWeight	stratum	sex	diagnosis	bodyPart	disp
0	150733174	7/11/2015	15.7762	V	Male	57	33	
1	150734723	7/6/2015	83.2157	S	Male	57	34	
2	150817487	8/2/2015	74.8813	L	Female	71	94	
3	150717776	6/26/2015	15.7762	V	Male	71	35	
4	150721694	7/4/2015	74.8813	L	Female	62	75	

2.3) Data imputation

In [23]:
```
# fillna
df['age'] = df['age'].fillna(df['age'].median())
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File c:\Users\lchan\OneDrive\Desktop\KMUTT\Y2T2\CPE232\.venv\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_loc(self, key)
   3811 try:
-> 3812     return self._engine.get_loc(casted_key)
   3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7096, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'age'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[23], line 2
      1 # fillna
----> 2 df['age'] = df[      ].fillna(df['age'].median())

File c:\Users\lchan\OneDrive\Desktop\KMUTT\Y2T2\CPE232\.venv\Lib\site-packages\pandas\core\frame.py:4113, in DataFrame.__getitem__(self, key)
   4111 if self.columns.nlevels > 1:
   4112     return self._getitem_multilevel(key)
-> 4113 indexer = self.columns.get_loc(key)
   4114 if is_integer(indexer):
   4115     indexer = [indexer]

File c:\Users\lchan\OneDrive\Desktop\KMUTT\Y2T2\CPE232\.venv\Lib\site-packages\pandas\core\indexes\base.py:3819, in Index.get_loc(self, key)
   3814     if isinstance(casted_key, slice) or (
   3815         isinstance(casted_key, abc.Iterable)
   3816         and any(isinstance(x, slice) for x in casted_key)
   3817     ):
   3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
   3820 except TypeError:
   3821     # If we have a listlike key, _check_indexing_error will raise
   3822     #  InvalidIndexError. Otherwise we fall through and re-raise
   3823     #  the TypeError.
   3824     self._check_indexing_error(key)

KeyError: 'age'
```

[Q1] From the above cell, Why it showing an error?

Ans: เพราะว่า Data frame ไม่มี column age ทำให้ไม่สามารถเข้าถึงได้จึงเกิด error ขึ้น

[Q2] Fix the error from Q1 problem.

In [24]:
```
# [Q2]

# hint: see the cell that run `df.pop()`
df["age"] = ages

# fillna again
df['age'] = df['age'].fillna(df['age'].median())

df.head()
```

Out[24]:

	caseNumber	treatmentDate	statWeight	stratum	sex	diagnosis	bodyPart	disp
0	150733174	7/11/2015	15.7762	V	Male	57	33	
1	150734723	7/6/2015	83.2157	S	Male	57	34	
2	150817487	8/2/2015	74.8813	L	Female	71	94	
3	150717776	6/26/2015	15.7762	V	Male	71	35	
4	150721694	7/4/2015	74.8813	L	Female	62	75	

2.4) Drop row that have missing value

In [50]:
```
# remove column by using .dropna()
df = df.dropna()
```

In [26]:
```
df.isnull().sum()
```

Out[26]:
```
caseNumber       0
treatmentDate    0
statWeight       0
stratum          0
sex              0
diagnosis        0
bodyPart         0
disposition      0
location         0
product          0
age              0
dtype: int64
```

Datetime

2.5) Working with the datetime format

In [51]:
```
df["treatmentDate"] = pd.to_datetime(df["treatmentDate"], format="%m/%d/%Y")
```

In [52]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 334837 entries, 0 to 334838
Data columns (total 14 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   caseNumber        334837 non-null  int64
 1   treatmentDate     334837 non-null  datetime64[ns]
 2   statWeight        334837 non-null  float64
 3   stratum           334837 non-null  object
 4   sex               334837 non-null  object
 5   diagnosis         334837 non-null  int64
 6   bodyPart          334837 non-null  int64
 7   disposition       334837 non-null  int64
 8   location          334837 non-null  int64
 9   product           334837 non-null  int64
 10  age               334837 non-null  int64
 11  Year              334837 non-null  int32
 12  Month             334837 non-null  int32
 13  newtreatmentDate  334837 non-null  object
dtypes: datetime64[ns](1), float64(1), int32(2), int64(7), object(3)
memory usage: 35.8+ MB
```

In [53]:
```
df['Year'] = df['treatmentDate'].dt.year
```

In [54]:
```
df['Month'] = df['treatmentDate'].dt.month
```

In [55]:
```
df.head()
```

Out[55]:

	caseNumber	treatmentDate	statWeight	stratum	sex	diagnosis	bodyPart	disp
0	150733174	2015-07-11	15.7762	V	Male	57	33	
1	150734723	2015-07-06	83.2157	S	Male	57	34	
2	150817487	2015-08-02	74.8813	L	Female	71	94	
3	150717776	2015-06-26	15.7762	V	Male	71	35	
4	150721694	2015-07-04	74.8813	L	Female	62	75	

[Q3] Can you change the format to DD/MM/YYYY? Show your work.

In [56]:
```
# write your code here
df['newtreatmentDate'] = df['treatmentDate'].dt.strftime('%d-%m-%Y')
df
```

Out[56]:

	caseNumber	treatmentDate	statWeight	stratum	sex	diagnosis	bodyPart
0	150733174	2015-07-11	15.7762	V	Male	57	33
1	150734723	2015-07-06	83.2157	S	Male	57	34
2	150817487	2015-08-02	74.8813	L	Female	71	94
3	150717776	2015-06-26	15.7762	V	Male	71	35
4	150721694	2015-07-04	74.8813	L	Female	62	75
...	...	...	...	...	...	...	..
334834	150739278	2015-05-31	15.0591	V	Male	59	76
334835	150733393	2015-07-11	5.6748	C	Female	68	85
334836	150819286	2015-07-24	15.7762	V	Male	71	79
334837	150823002	2015-08-08	97.9239	M	Female	59	82
334838	150723074	2015-06-20	49.2646	M	Female	57	34

334837 rows × 14 columns

Combine Dataframe by .merge and .concat

2.6 Merge

In [33]:
```
import pandas as pd

superstore_order = pd.read_csv('Superstore/superstore_order.csv')
superstore_people = pd.read_csv('Superstore/superstore_people.csv')
superstore_return = pd.read_csv('Superstore/superstore_return.csv')
```

In [34]:
```
superstore_order.columns
```

Out[34]:
```
Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
       'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
       'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
       'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
      dtype='object')
```

In [35]:
```
superstore_order.merge(superstore_return[superstore_return["Returned"]=="Yes"],
 on="Order ID" ,
 how="inner")\
 [["Customer ID", "Returned"]]\
 .drop_duplicates()
```

Out[35]:

	Customer ID	Returned
0	ZD-21925	Yes
3	TB-21055	Yes
10	JS-15685	Yes
13	LC-16885	Yes
20	BS-11755	Yes
...	...	...
688	ED-13885	Yes
689	TS-21205	Yes
696	MF-17665	Yes
702	SH-19975	Yes
705	RB-19435	Yes

222 rows × 2 columns

[Q4] What does the argument `how="inner"` do?

Ans: **เป็นการทำให้เลือกตัวที่** intersection **กันระหว่าง** 2 dataframes

[Q5] In your opinion, what information that the result above conveys?

Ans: **ข้อมูลข้างบนเป็นการบอกว่ามี** Customer ID **คนไหนที่คืนสินค้ากลับมาบ้าง**

2.7) Concatenate

In [36]:
```
pd.concat([superstore_order, superstore_people], axis=1, join='inner')
```

Out[36]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Co
0	1	CA-2016-152156	08/11/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	U
1	2	CA-2016-152156	08/11/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	U
2	3	CA-2016-138688	12/06/2016	16/06/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	U
3	4	US-2015-108966	11/10/2015	18/10/2015	Standard Class	SO-20335	Sean ODonnell	Consumer	U

4 rows × 23 columns

[Q6] What is the difference between `inner` and `outer` on parameter `join` in `pd.concat`?
Ans ถ้า dataframe ที่จะมา pd.concat กัน inner จะเป็นการ intersection เอาเฉพาะข้อมูลที่มีในทั้ง 2 เท่านั้น ส่วน outer จะเป็นการ union จะเป็นการเอาข้อมูลมาทั้งหมด

[Q7] Append new record at the end of superstore_order. Set the 'Row ID' to your student ID and mock other required fields based on existing entries.

In [37]:
```
# write your code here
from datetime import datetime
newdata = {
    'Row ID' : 67070501059,
    "Order ID" : "TH-2015-107944",
    "Order Date" : "23/03/2015",
    "Ship Date"  : "25/03/2015",
    "Ship Mode" : "First Class",
    "Customer ID" : "CL-10362",
    "Customer Name" : "Chanon Chanon",
    "Segment" : "Consumer",
    "Country" : "Thailand",
    "City" : "Kanchanaburi",
    "State" : "Thamaka",
    "Postal Code" : 71120,
    "Region" : "West",
    "Product ID" : "TEC-MA-10004626",
    "Category" : "Technology",
    "Sub-Category" : "Machines",
```

```
    "Product Name" : "Lexmark 20R1285 X6650 Wireless All-in-One Printer",
    "Sales" : 480.0,
    "Quantity" : 4,
    "Discount" : 0.00,
    "Profit" : 225.6
}
new_df = pd.DataFrame([newdata])
superstore_order = pd.concat([superstore_order, new_df], ignore_index=True)
```

In [38]:
```
print(superstore_order.iloc[-1])
```

```
Row ID                                                67070501059
Order ID                                          TH-2015-107944
Order Date                                            23/03/2015
Ship Date                                             25/03/2015
Ship Mode                                           First Class
Customer ID                                           CL-10362
Customer Name                                     Chanon Chanon
Segment                                               Consumer
Country                                               Thailand
City                                                Kanchanaburi
State                                                  Thamaka
Postal Code                                              71120
Region                                                    West
Product ID                                      TEC-MA-10004626
Category                                             Technology
Sub-Category                                           Machines
Product Name     Lexmark 20R1285 X6650 Wireless All-in-One Printer
Sales                                                    480.0
Quantity                                                     4
Discount                                                   0.0
Profit                                                   225.6
Name: 8880, dtype: object
```

Groupby

In [39]:
```
superstore_order["Profit Ratio"] = superstore_order["Profit"]/superstore_order["

superstore_order.groupby(["Category", "Sub-Category"]).agg(mean_profit_ratio = (
```

Out[39]:

Category	Sub-Category	mean_profit_ratio
Furniture	**Bookcases**	-0.127756
	Chairs	0.045028
	Furnishings	0.140782
	Tables	-0.147916
Office Supplies	**Appliances**	-0.145513
	Art	0.251678
	Binders	-0.191641
	Envelopes	0.421913
	Fasteners	0.301157
	Labels	0.429984
	Paper	0.425586
	Storage	0.092382
	Supplies	0.104970
Technology	**Accessories**	0.219012
	Copiers	0.317826
	Machines	-0.054632
	Phones	0.118926

[Q8] Describe an information that the result above conveys?

Ans:ดูว่าสินค้าประเภทไหนได้ค่าเฉลี่ยกำไรเป็นอัตราส่วนเท่าไหร่

Pivot and Melt

Pivot

In [40]:
```
superstore_order.pivot_table(index="State", columns="Ship Mode", values="Order I
```

Out[40]:

Ship Mode State	First Class	Same Day	Second Class	Standard Class
Alabama	9.0	1.0	18.0	30.0
Arizona	42.0	15.0	22.0	123.0
Arkansas	10.0	2.0	8.0	35.0
California	302.0	106.0	346.0	1000.0
Colorado	43.0	5.0	32.0	95.0
Connecticut	19.0	8.0	11.0	39.0
Delaware	16.0	2.0	13.0	55.0
District of Columbia	0.0	0.0	3.0	7.0
Florida	47.0	25.0	57.0	210.0
Georgia	19.0	15.0	31.0	108.0

In [41]:
```
pivot_table_result = superstore_order.pivot_table(index="State", columns="Ship M
print(pivot_table_result)
```

Ship Mode	First Class	Same Day	Second Class	Standard Class
State				
Alabama	9.0	1.0	18.0	30.0
Arizona	42.0	15.0	22.0	123.0
Arkansas	10.0	2.0	8.0	35.0
California	302.0	106.0	346.0	1000.0
Colorado	43.0	5.0	32.0	95.0
Connecticut	19.0	8.0	11.0	39.0
Delaware	16.0	2.0	13.0	55.0
District of Columbia	0.0	0.0	3.0	7.0
Florida	47.0	25.0	57.0	210.0
Georgia	19.0	15.0	31.0	108.0
Idaho	3.0	0.0	2.0	13.0
Illinois	58.0	24.0	96.0	249.0
Indiana	13.0	3.0	30.0	79.0
Iowa	1.0	1.0	4.0	17.0
Kansas	6.0	1.0	2.0	15.0
Kentucky	12.0	5.0	49.0	62.0
Louisiana	7.0	2.0	14.0	15.0
Maine	0.0	0.0	0.0	5.0
Maryland	18.0	7.0	12.0	63.0
Massachusetts	14.0	4.0	35.0	71.0
Michigan	20.0	16.0	43.0	151.0
Minnesota	9.0	4.0	13.0	59.0
Mississippi	3.0	4.0	7.0	36.0
Missouri	7.0	2.0	20.0	24.0
Montana	1.0	1.0	0.0	13.0
Nebraska	6.0	3.0	6.0	20.0
Nevada	4.0	1.0	12.0	17.0
New Hampshire	2.0	0.0	10.0	13.0
New Jersey	5.0	1.0	20.0	87.0
New Mexico	1.0	0.0	9.0	22.0
New York	155.0	57.0	183.0	606.0
North Carolina	36.0	14.0	40.0	139.0
North Dakota	0.0	0.0	5.0	2.0
Ohio	66.0	47.0	84.0	199.0
Oklahoma	5.0	6.0	7.0	44.0
Oregon	20.0	0.0	15.0	81.0
Pennsylvania	103.0	9.0	78.0	341.0
Rhode Island	16.0	0.0	21.0	16.0
South Carolina	3.0	5.0	18.0	16.0
South Dakota	2.0	0.0	0.0	9.0
Tennessee	21.0	2.0	24.0	118.0
Texas	125.0	37.0	161.0	537.0
Thamaka	1.0	0.0	0.0	0.0
Utah	4.0	2.0	19.0	28.0
Vermont	0.0	0.0	1.0	2.0
Virginia	39.0	4.0	33.0	115.0
Washington	56.0	34.0	97.0	265.0
West Virginia	0.0	0.0	0.0	3.0
Wisconsin	12.0	3.0	10.0	66.0
Wyoming	0.0	0.0	0.0	1.0

Melt

In [42]:
```
melted_result = pd.melt(pivot_table_result.reset_index(), id_vars=["State"], var
print(melted_result)
```

```
            State       Ship Mode  Order Count
0         Alabama     First Class          9.0
1         Arizona     First Class         42.0
2        Arkansas     First Class         10.0
3      California     First Class        302.0
4        Colorado     First Class         43.0
..             ...             ...          ...
195      Virginia  Standard Class        115.0
196    Washington  Standard Class        265.0
197  West Virginia  Standard Class          3.0
198     Wisconsin  Standard Class         66.0
199       Wyoming  Standard Class          1.0

[200 rows x 3 columns]
```

[Q9] What is the advantage of using `melt` ?

Ans: **ใช้ในการ** Reshape **ข้อมูลจาก** wide format **เป็น** long format **ทำให้สามารถอ่านได้ง่ายขึ้น**

[Q10] From the superstore_order, display the ascending order considering values in the 'Profit' column to group the 'Category'.

In [43]:
```
#enter your code
superstore_order.groupby("Category")["Profit"].sum().sort_values(ascending=True)
```

Out[43]:
```
Category
Furniture          16858.5619
Office Supplies   105827.0238
Technology        133636.0932
Name: Profit, dtype: float64
```

[Q11] Create a new column that calculates the total price (sale*quantity) before discount then group by 'product id' and 'category', then show the mean of the total price

In [44]:
```
#enter your code here
superstore_order['Total Price'] = superstore_order['Sales'] * superstore_order['
result = superstore_order.groupby(['Product ID', 'Category'])['Total Price'].mea
print(result)
```

```
Product ID       Category
FUR-BO-10000112  Furniture     7426.566000
FUR-BO-10000330  Furniture     1258.192000
FUR-BO-10000362  Furniture     1726.898000
FUR-BO-10000468  Furniture      426.532400
FUR-BO-10000711  Furniture     3194.100000
                                   ...
TEC-PH-10004912  Technology     747.320000
TEC-PH-10004922  Technology     673.249500
TEC-PH-10004924  Technology      57.149333
TEC-PH-10004959  Technology     412.009000
TEC-PH-10004977  Technology    2441.475429
Name: Total Price, Length: 1846, dtype: float64
```

[Q12] Complete the function to apply `ratio` column that calculates from `First Class` and `Standard Class` columns on `pivot_table_result`

In [45]:
```
pivot_table_result.head(1)
```

Out[45]:

Ship Mode	First Class	Same Day	Second Class	Standard Class
State				
Alabama	9.0	1.0	18.0	30.0

In [46]:
```
# function to transform the ratio
def get_class_ratio(row):

    # get the first class column
    first_class = row['First Class']

    # get the standard class column
    standard_class = row['Standard Class']

    # calculate the ratio
    ratio = first_class /standard_class

    return ratio

pivot_table_result["ratio"] = pivot_table_result.apply(get_class_ratio, axis=1)

pivot_table_result.head()
```

```
C:\Users\lchan\AppData\Local\Temp\ipykernel_10084\3439814514.py:11: RuntimeWarnin
g: divide by zero encountered in scalar divide
  ratio = first_class /standard_class
```

Out[46]:

Ship Mode	First Class	Same Day	Second Class	Standard Class	ratio
State					
Alabama	9.0	1.0	18.0	30.0	0.300000
Arizona	42.0	15.0	22.0	123.0	0.341463
Arkansas	10.0	2.0	8.0	35.0	0.285714
California	302.0	106.0	346.0	1000.0	0.302000
Colorado	43.0	5.0	32.0	95.0	0.452632

[Q13] After complete Q13, What does the `apply` function do?

Ans: เป็นการเอาฟังก์ชันที่เราเขียนไปใช้กับ Data frames

[Q14] Create a new column(`short_ratio`) that works the same as Q12 but with `lambda` function

In [47]:
```
pivot_table_result["short_ratio"] = pivot_table_result.apply(lambda row: row['Fi

pivot_table_result.head()
```

```
C:\Users\lchan\AppData\Local\Temp\ipykernel_10084\2366408109.py:1: RuntimeWarnin
g: divide by zero encountered in scalar divide
  pivot_table_result["short_ratio"] = pivot_table_result.apply(lambda row: row['F
irst Class']/row['Standard Class'], axis=1)
```

Out[47]:

Ship Mode State	First Class	Same Day	Second Class	Standard Class	ratio	short_ratio
Alabama	9.0	1.0	18.0	30.0	0.300000	0.300000
Arizona	42.0	15.0	22.0	123.0	0.341463	0.341463
Arkansas	10.0	2.0	8.0	35.0	0.285714	0.285714
California	302.0	106.0	346.0	1000.0	0.302000	0.302000
Colorado	43.0	5.0	32.0	95.0	0.452632	0.452632

[Q15] What is the difference between `apply` and `lambda` function? give 2 examples use case.

Ans: apply เป็นมธอดที่ใช้เรียกใช้ฟังก์ชันใน Data frame โดยจะนำฟังก์ชันนั้นไปประมวลผลกับแต่ละ element ส่วน lambda เป็นการ define ฟังก์ชันใหม่ขึ้นมาโดยฟังก์ชันนั้นไม่จำเป็นต้องมีชื่อสำหรับฟังก์ชันง่าย ๆ โดยไม่จำเป็นต้องใช้ def

Use case 1

ใช้ apply กับ function str.upper เพื่อทำให้ตัวอักษรทั้งหมดใน Data frame column State เป็นตัวใหญ่ทั้งหมด มาอยู่ใน column State_Upper

In [64]:
```
melted_result["State_Upper"] = melted_result["State"].apply(str.upper)
melted_result.head()
```

Out[64]:

	State	Ship Mode	Order Count	State_Upper
0	Alabama	First Class	9.0	ALABAMA
1	Arizona	First Class	42.0	ARIZONA
2	Arkansas	First Class	10.0	ARKANSAS
3	California	First Class	302.0	CALIFORNIA
4	Colorado	First Class	43.0	COLORADO

Use case 2

ใช้ lambda ในการสร้างฟังก์ชันในการบวก Ship Mode ทั้งหมดเข้าด้วยกัน แล้วนำมาใช้ใน apply

In [ ]:
```
pivot_table_result["total shipping"] = pivot_table_result.apply(lambda row: row[
pivot_table_result.head()
```

Out[ ]:

Ship Mode State	First Class	Same Day	Second Class	Standard Class	ratio	short_ratio	total shipping
Alabama	9.0	1.0	18.0	30.0	0.300000	0.300000	58.0
Arizona	42.0	15.0	22.0	123.0	0.341463	0.341463	202.0
Arkansas	10.0	2.0	8.0	35.0	0.285714	0.285714	55.0
California	302.0	106.0	346.0	1000.0	0.302000	0.302000	1754.0
Colorado	43.0	5.0	32.0	95.0	0.452632	0.452632	175.0